

Learning to Drive by Imitation: An Overview of Deep Behavior Cloning Methods

Abdoulaye O. Ly  and Moulay Akhloufi , *Senior Member, IEEE*

Abstract—There is currently a huge interest around autonomous vehicles from both industry and academia. This is mainly due to recent advances in machine learning and deep learning, allowing the development of promising methods for autonomous driving. The gap toward full autonomy is incrementally being reduced with essentially three main existing approaches. First, Modular systems that combine a pipeline of methods with each solving one specific sub-task of driving. Second, Direct Perception techniques that directly estimate affordances (car orientation, distances between lane borders, etc) used to compute control commands through a simple logic. Finally, end-to-end frameworks that automatically map raw sensor data to actuation values. The objective of this paper is to review some recent works focusing on end-to-end deep learning models for lane stable driving, as well as some publicly available real world datasets and open-source simulators that enable the development and evaluation of such methods.

Index Terms—Lane stable driving, autonomous vehicles, autonomous driving, end-to-end driving, vehicle control prediction, driving datasets, driving simulators, deep learning.

I. INTRODUCTION

CONSIDERING the potential benefits of autonomous vehicles such as increasing safety in roads, giving humans more free time, reducing transportation costs, decreasing pollution and more [1], the development of autonomous cars which are capable of fully driving without human intervention has been an increasingly interesting area that researchers have been studying for several decades. Many pioneering works were achieved during the 80 s. First, Dickmanns *et al.* [2] turned a Mercedes-Benz van into a autonomous car that drove itself up to 96 km/h on more than 20 km road without traffic based on what the authors called dynamic vision. In 1988, Pomerleau developed ALVINN [3], the first autonomous vehicle based on a feed-forward neural network receiving inputs from a video camera and a rangefinder.

Between 2004 and 2017, the first competitions for self-driving cars were held by DARPA primarily in the Mojave Desert and then in an urban environment. A participant in the off-road competition nicknamed DAVE [4], inspired by ALVINN, was developed using an end-to-end learning approach. The urban

challenge has seen three finalists [5]–[7] complete the 52 miles long track without human intervention, giving the first signal that autonomous driving may be a realistic goal even in an urban setting.

Nowadays, the interest around self-driving cars has just exploded, particularly in the industry. The race towards fully autonomous vehicles involving car makers and technology companies reached its peak. Numerous announcements have been made regarding the deployment or testing of these cars. In 2014, Tesla released Autopilot believed to have a level between 2 and 3 in the SAE scale of driving automation [8]. In addition Waymo, Uber to name just a few carried out millions of self-driving tests on public roads [9]. This shows that autonomous cars are closer to becoming a reality.

There are three main approaches for autonomous driving (Fig. 1). 1. Modular systems [10], [11] that combine a pipeline of various highly engineered models with each focusing in just one specific task such as segmentation, object detection, object tracking, localization, route planning, vehicle control, etc. 2. Direct Perception techniques [12], [13] that directly estimates affordances (car orientation, distance between lane borders, distances between the agent and other dynamic objects such as cars or pedestrian taking part in traffic, etc.) via a perception module which is generally a deep network. The affordances are then used to compute the actuation values through a simple hand crafted algorithm. 3. End-to-end methods that automatically maps raw pixels and/or other visual data from a Light Detection and Ranging (LiDAR) sensor, events or fisheye cameras, etc. to predict control commands.

Knowing that self-driving technology is a broad and fertile field which can literally reshape the way transportation is currently done, we just limit the scope of this review to end-to-end deep imitative vision based techniques for driving. Indeed, we exclude reinforcement learning [14]–[18], approaches which work through trial and error and are essentially developed under simulation due to the high cost and high risks of damages involved. The methods presented in this paper are usually referred as behavior cloning methods in the literature as they transform the driving task into a supervised learning problem.

In the past, various surveys have been done in the field of self-driving vehicles. Tai *et al.* [19] review control, manipulation and navigation techniques based on deep imitative and reinforcement learning for both indoor and outdoor robots. In [20], the focus is put on algorithms for motion planning and low level control of autonomous vehicles. Badue *et al.* [21] survey perception, localization, and decision making methods for autonomous

Manuscript received December 18, 2019; revised May 15, 2020; accepted June 7, 2020. Date of publication June 15, 2020; date of current version May 25, 2021. (Corresponding author: Abdoulaye O. Ly.)

The authors are with the Perception, Robotics, and Intelligent Machines Research Group, Department of Computer Science, Université de Moncton, Moncton, NB E1A 3E9, Canada (e-mail: ablayelyfondou@gmail.com; moulay.akhoulfi@umoncton.ca).

Color versions of one or more of the figures in this article are available online at <https://ieeexplore.ieee.org>.

Digital Object Identifier 10.1109/TIV.2020.3002505

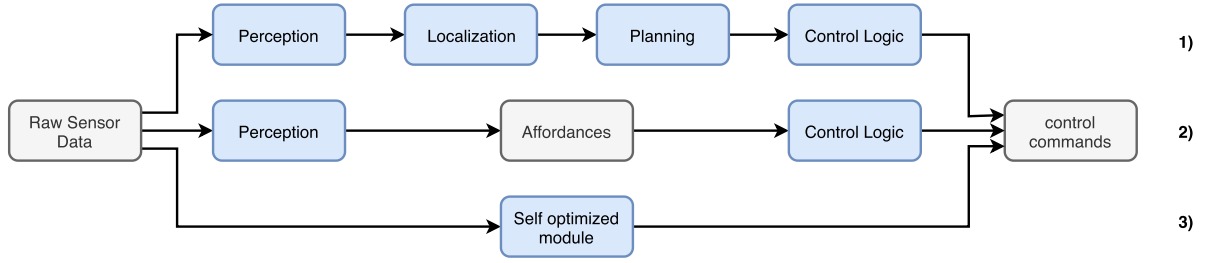


Fig. 1. Three main approaches for autonomous driving: 1. A modular pipeline framework that combines various machine learning and hand crafted algorithms with each solving a specific sub-task, 2. Direct perception which consist of estimating affordances first via a perception module and then predict actuation values using a simple control logic, 3. End-to-end models that output directly control commands from raw data inputs using one or more self optimized modules.

TABLE I
PERCEPTUAL DATA TYPES OF VARIOUS PUBLIC DATASETS FOR END-TO-END AUTONOMOUS DRIVING MODELS

Name	Grayscale	RGB	Point clouds	Events
Comma.ai [28]		✓		
The Comma2k19 Dataset [29]		✓		
BDDV [30]		✓		
The Oxford RobotCar dataset [31]		✓	✓	
BDD [32]		✓	✓	
LiDAR Video Driving Datasets [33]		✓	✓	
DDD17 [34]	✓			✓
The Kitti dataset [35]	✓	✓	✓	

driving. Campbell *et al.* [22] provides an overview of approaches, challenges and opportunities of autonomous driving in urban environments. Katrakazas *et al.* [23] reviewed state of the art motion planning methods for on-road self-driving cars. Xue *et al.* [24] discuss existing scene understanding approaches with a focus on predicting intention of other traffic participants. In a recent survey, Kang *et al.* [25] presented numerous public driving datasets, as well as open and commercial virtual testing environments.

Considering the lack of reviews targeting end-to-end deep methods for autonomous driving and numerous literature being published recently regarding the topic, this paper will address the problem by providing the following key contributions:

- A presentation of some existing real word datasets for continuous driving, and the introduction of various simulators ranging from those with high level to low level of details. Our focus will be directed toward publicly available datasets and simulators, or those that are intended to be shared with the community.
- A description of a wide range of end-to-end deep learning based approaches. By end-to-end, we mean supervised methods that map raw visual inputs to low level (steering angle, speed, etc) or high level (driving path, driving intention, etc.) of actuation commands using almost only deep networks.
- A detailed qualitative and quantitative comparison and discussion of the methods.

The reminder of the paper is organized as follows. Section II surveys some existing datasets freely available to the community. Section III presents 10 interesting public simulators for the development of end-to-end driving models. Section IV reviews various end-to-end imitative models subdivided into 5 main categories. Section V provides a discussion of some challenges and opportunities related to end-to-end autonomous driving. Finally, Section VI provides a summary and conclusion of the paper.

II. DATASETS

In this section, we present some publicly available datasets relevant to the development of end-to-end vision approaches for predicting control commands in the context of autonomous driving. 8 popular datasets, ordered following the number of perceptual data supported, are introduced. An overview of available perceptual data for each dataset is provided in Table I. Datasets focused on object detection or segmentation ([26], [27]) alone are not included here since they are beyond the scope of this paper.

Comma.ai Driving Dataset [28] was released in 2016 by Comma.ai, a startup promoting and working on autonomous vehicle technology using just very accessible sensors such as those integrated in today's smartphones. The dataset consists of a total of 7.25 hours of driving spread into 11 videos. The videos were recorded from a Point grey camera placed on an Acura ILX

2016. The frames, captured at a rate of 20 Hz, are released with a resolution of 160×320 pixels. The dataset available in HDF5 format, a popular format in machine learning, also includes raw data from other sensors such as global positioning system (GPS), gyroscope, inertial measurement unit (IMU), LiDAR and radar. Example of interesting ground truths for end-to-end driving are steering angle and car speed.

The Comma2k19 Dataset [29] was proposed recently by Comma.ai. The dataset is constructed of 2019 one minute long video sequences from a single road facing camera, as well as data from sensors like GPS, thermometers and an IMU. Instead of providing computed value from the phone GPS, the authors provide raw Global Navigation Satellite System (GNSS) data for which they proposed an open source processing library named Laika. The dataset also includes logs of steering angle, radar and wheel speed. For data collection, the authors opted for a fixed 20 km long segment of California 280 highway. This choice is justified by the desire to have enough dense data, or in other words several traversals of the same road with different conditions making the dataset very robust for driving algorithms benchmarking.

Berkeley DeepDrive Video Dataset (BDDV) [30] is a large scale dataset suitable for developing generic driving models. The dataset comprises more than 10 k hours of videos from a front-facing road camera. Each sequence contains approximately 40 seconds of real world driving. Along those visual data, BDDV comes with raw or preprocessed data such as car speed and course from different others sensors including GPS, IMU, gyroscope and magnetometer. The dataset is very diverse and covers various driving scenes: urban, highway and country roads. Additionally, different weather (snowy, sunny, rainy) and lighting (daytime, nighttime etc) conditions are available in BDDV making it very complete compared to other proposed datasets. Recently, the datasets was extended to include over 100 K of high definition videos captured at a rate of 30fps from 3 US cities: New York, Berkeley and San Francisco [27]. Besides end-to-end generic driving models, the dataset is also annotated so that it can also be used for object detection, semantic instance segmentation and more.

The Oxford RobotCar Dataset [31] is a very large dataset collected in 2014 and 2015 provided by the University of Oxford. It consists of 20 millions 1024×1024 images captured from 6 cameras positioned on a Nissan LEAF. Along the RGB frames, other measurements were captured using a LiDAR sensor, a GPS and an Inertial Navigation System (INS). The LiDAR is responsible for producing the 3D pointclouds, the GPS gives the global positioning coordinates defined by longitude, latitude, altitude and uncertainty. The association of GPS and INS allows the interpolation of values for velocity, 3D Universal Transverse Mercator (UTM) coordinates and attitude. The data collection required 1000 km of travelled distance in urban areas in which some sections of road were traversed several times making the dataset very interesting and challenging for navigation. To make it easy to work with, the authors furnished a software development toolkit on top of the dataset which can return a map of the reconstructed pointclouds effortlessly.

Baidu Driving Dataset (BDD) [32] comes from the technology company Baidu via its Street View project. 10 000 km of recordings spanning from March to November 2016 were required to collect 10 TB of raw data from multiple cities across China. The data collection platform was equipped with 3 monocular cameras capturing 2448×2048 RGB images at a rate of 8 frames per second each. The positioning of the cameras is as follows: one facing the road ahead and two on the sides (left and right). In addition to the frames, the dataset includes 3D pointclouds data provided by a 32 beam LiDAR. The platform was also equipped with a high precision (3 cm) GNSS/IMU module logging position and attitude of the vehicle at a rate of 20 times per second. From the GNSS/IMU module recordings, ground truths for lateral control and longitudinal control can be approximated.

LiDAR Video Driving Datasets (LiVi-Set) [33] is among the first datasets that proposes 2D and depth images intended for driving policy learning (steering angle and speed). It contains 100 km (or 20 hours) of 3D point clouds data, making it the largest public LiDAR dataset for driver behavior learning. An interesting fact about LiVi-Set is that the 3D points clouds data are already pre-processed and available in the standard LAS format. The dataset is also very representative of urban scenarios ranging from low to high traffic situations. Indeed, its diversity is illustrated by the statistics: 1500 cars, 500 roads signs, 160 traffic lights, 363 crossroads and 32 footbridges. Additionally, the collected data is of very good quality with a 70 meters ranging for depth along with 2 cm in resolution from the LiDAR sensor. Moreover, the resolution of the RGB images from the digital camera is 1920×1080 pixels.

DAVIS Driving Dataset (DDD17) [34] is the first public dataset bringing together event-based vision and traditional frames for end-to-end autonomous driving proposed by the Institute of Neuroinformatics Zurich in 2017. Both the events and the standard frames were captured with the same DAVIS 346B prototype camera at a resolution of 346×260 pixels. The recording time totals over 12 hours. The visual frames are associated with ground truths from several other sensors such as GPS, INS to name just a few. The GPS gives latitude and longitude coordinates while the INS helps collect the vehicle speed and orientation. Additionally, the steering angle ranging from -600 to $+600$, the position of the accelerator pedal, the position of the transmission gear, the status of the brake pedal and more are provided along with the visual data. The dataset covers city and highway driving in various scenarios including daytime, evening and night. It also contains driving sequences of both wet and dry weather conditions.

The Kitti Dataset [35], made from a collaboration between Karlsruhe Institute of Technology in Germany and Toyota Technological Institute at Chicago, is one the most popular dataset in the context of autonomous driving. It consists of 6 hours of data collected using a Volkswagen station wagon and cover a wide range of traffic conditions such as inner cities, rural areas or high way driving. Data were recorded from different sensors including a Velodyne LiDAR rotating at 10 frames per second and 100 k point-clouds per frame, color and grayscale stereo

TABLE II
COMPARISON OF VARIOUS SIMULATORS FOR END-TO-END AUTONOMOUS DRIVING MODELS

Name	Underlying Platform(s)	Graphics Quality	Functional fidelity	Client API	Supported Operating Systems
Carla [37]	Unreal Engine 4	High	High	Python	Windows / Linux
Airsim [38]	Unreal Engine 4	High	High	Python / C++	Windows / Linux
Autovi-sim [39]	N/A	Low	High	N/A	N/A
Sim4CV [40]	Unreal Engine 4	High	Medium	Python / C++ / Matlab	Windows
Gazebo [41]	ODE, Bullet, Simbody, Dart	Medium	Medium	python / C++	Linux
Udacity [42]	Unity	Medium	Low	Python	Windows / Linux / Mac
DeepGTAV [43]	RockStar Advanced Game Engine	High	Medium	Python	Windows
TORCS [44]	PLIB	Medium	Medium	C++, Java, Python	Linux
Europilot [45]	Prism3D	High	Low	Python	Linux
VISTA [46]	N/A	High	Low	N/A	N/A

cameras with a resolution of 1.4 Megapixels, a high precision inertial and GPS navigation system, etc. Kitti is also characterized by its multi-purpose aspect. Indeed ground truths are provided for object detection, segmentation, Simultaneous Localization And Mapping (SLAM), tracking and more. Additionally, the dataset contains lateral control commands such as speed and acceleration along with the raw sensors, data.

III. SIMULATORS

Conducting real world experimentations are expensive and can cost up to US \$1 million per vehicle [36] when parameters such as personal and infrastructure maintenance, insurance, to name just a few are taken into account. Such amount of money is not within the reach of everybody and can limit the development of autonomous vehicles. Besides that, it is even more problematic to test driving models in dangerous situations which may lead to the harm or death of humans in the physical world. In that, simulation is vital for self-driving cars development and testing. Many simulators ranging from low level to high level of details have been proposed in the literature. We deliberately omit proprietary simulation softwares such as Carsim and Prescan and favor open-source (or planned to be released as such by authors) simulators, which are mostly extensible and allow a democratization of research in the field of autonomous vehicles. A comparison is provided in Table II to give some insights in the characteristics of each of the presented simulators.

CARLA [37] is an open source simulator for self-driving cars based on Unreal Engine 4 born out of a joint initiative of researchers from Intel Labs, Toyota Research Institute and the Computer Vision Center of Barcelona. It provides a diverse and realistic urban environment which initially consisted of 40, 16 and 50 building, vehicle and pedestrians models, respectively. Additionally, the simulator, at the time it was first published, could be adjusted to 9 weather conditions and 2 degrees of illumination such as mid-day and sunset. The simulator has a Server-Client architecture where the server gives information

from the current environment and the client controls the car agent. Supported sensors include cameras, GPS, compass and 3D sensors. 3D data are accessible in a pre-processed and fused form. The information provided by the simulator is composed of RGB, depth and semantic images (example of classes are road lane marking, sidewalk, car, pedestrian, building, etc.), position and car orientation, position of all dynamic objects around the agent, vehicle speed and steering angle, etc. Among the control commands that can be triggered from the python client API are acceleration, brake and steering.

Airsim [38], like Carla, is also a very realistic open source simulator based on Unreal Engine 4 developed by Microsoft. While it was initially intended for Unmanned Aerial Vehicles (UAVs) simulation only, the authors recently introduced the support for car agents. With Airsim, the goal is to bridge the gap between reality and simulation by proposing a high fidelity simulation of the real world. It is designed to be flexible and easily extensible, in a way that it can be added to any Unreal Engine project, or even integrated with Unity projects. There are already many rich generated environments contributed by Microsoft and the broader community available through the simulator marketplace. Many sensors used in the industry for autonomous vehicles such as LiDAR, GPS, IMU, cameras, etc. are offered. Unlike Carla, which return processed and fused depth data, Airsim provides raw 3D point clouds via its LiDAR API, making it more challenging for computer vision applications. Airsim also supports multi-vehicle capability enabling the testing of multi-agent driving algorithms. Another interesting feature provided by the authors is the No-display mode that allows a high frame rate of video recording by turning off the visual rendering. This feature can be especially useful during training of driving models.

Autovi-sim [39] is a high level of details autonomous vehicle simulator proposed recently. It is modular and extensible, but also has a very flexible simulation environment allowing varying configurable vehicles, sensors, traffic density and weather conditions. The simulator is divided into 8 modules : Roads,

Infrastructure, Road Network, Environment, Vehicles, Non-Vehicle Traffic, Drivers, and Analysis. Road module describes the roads structure, hazards and occupancy. Road Network informs about current and future road traffic. Environment module allows researchers or engineers to specify weather effects and time of the day. Infrastructure module defines the positioning and state of the elements of the road such as traffic lights and signage. Non-Vehicle Traffic module controls cyclists and pedestrians behavior and destination. Dangerous situations can be simulated using this module to test driving algorithms in front of unusual pedestrian behaviors. Driver modules consist of three components: Vehicles defining the car model and the configuration of sensors, Control and Dynamics providing information about brake inputs, throttle, and steering as well as vehicle dynamics such as slipping for instance, Perception describing the agent surrounding environment with information like detection time, classification error rate and more. Analysis module returns information regarding velocities, positions and behaviors of other vehicles, cyclists and pedestrians participating in the traffic. The simulator is suitable for the development of both modular and end-to-end deep driving models.

Sim4CV [40] is an end-to-end high fidelity simulator based on Unreal Engine 4 furnishing rich and realistic environments. It supports both UAVs and ground vehicles. Three modes are currently available: autonomous driving, autonomous flying and aerial object tracking. The authors also propose APIs for various programming languages including Python, C++ and Matlab. Additionally, a marketplace is provided along with the simulator in order to centralize all assets contributed by the community. Sim4CV comes also with a friendly visual interface making it easy to create custom maps for instance. Currently supported sensors include only RGB cameras but the authors plan to add depth sensors such as LiDAR and radar in the future. The car agent can be controlled through steering angle, accelerator and brake as commonly proposed by other simulators.

Gazebo [41] is a general purpose mobile robot simulator provided by the University of Southern California. Its development started in early 2000s and since, the simulator has been greatly enriched due to its active community of contributors. It currently supports 4 physics engine (ODE, Bullet, Simbody and Dart) and is well integrated with the Robotic Operating System (ROS), a popular library for robot programming. Gazebo is open and proposes several layers of abstraction that makes it easier to work with and extend. In 2017, vehicle and city simulator (citysim) is proposed for the development of autonomous driving algorithms in an urban city. The dynamic environments comprise animated pedestrians, stop lights and signals. Additionally, new elements such as terrain, water, sky, and particles effects can be added to the simulation world, or modify the existing ones. Fifteen sensors for noise models, 2D and visualizations are already available. Gazebo also provides a real-time tool for plotting sensor data. Moreover, an important feature of the simulator is that control interfaces can be extended via plugins.

Udacity Self-Driving Car Simulator (USCS) [42] is provided by Udacity for its self-driving car nanodegree program in collaboration with Nvidia and Unity. The simulator is based on Unity engine and the source code is publicly available on Github.

3 cameras (left, center, and right) are positioned at the front of the car agent and are the only source of data for perception. The remaining information provided by the simulator are speed, brake, throttle, and steering angle. However, the simulator is basic compared to Airsim or Carla and is mostly used for educational purpose. Additionally, the lack of documentation and can make it challenging to exploit or extend.

DeepGTAV [43] is a plugin developed using the programming language C++ to transform the popular video game Grand Theft Auto 5 (GTA 5) into a research simulator for autonomous driving. GTA 5 is known for its realistic, huge and diverse scenes ranging from countryside to city environments, day to night times, winter to summer period, etc. making it an interesting simulator for driving models development and testing. The plugin offers a python library to control steering, velocity, throttle and brake, define the initial conditions of the simulation and collect a bunch of raw sensor data from the environment. Data includes location, time, weather, RGB frames and more.

The Open Racing Car Simulator (TORCS) [44] is an adapted 3D video game for evolutionary and computational intelligence algorithm benchmarking. TORCS is very popular among researchers and many proposed reinforcement learning models for driving have been trained and tested on it. TORCS has various tracks and a robot agent has the ability to compete with other robot drivers for performance comparison. Numerous information from different sensors are provided via its client APIs including the angle between the direction of the track and the direction of the car, the elapsed time in the current lap, the accumulated damages of the vehicle, the distance covered along track line, the distance between the car and the track edges, the car speed, the position of the gear, steering values and more. In its original version, the simulator is provided with C++ and Java client APIs. An adaptation to OpenAI gym named gym-TORCS enables Python usage and also extends the logged sensor data with image recordings.

Europilot [45], similarly to DeepGTAV, is a project that transforms the open source video game Euro Truck Simulator 2 (ETS 2) into a self-driving car simulator. ETS 2 is based on the Prim3D game engine and has better graphics quality than TORCS or Udacity. The three main environments in ETS 2 are country road, mountain road and freeway. Europilot supports the Python programming language and is only tested on Linux. However, it should be arguably easy to adapt to other operating systems since ETS 2 runs on both Windows and Mac. States provided by the plugin include a screenshot image of the front facing road and the values for brake, gas and gears.

Virtual Image Synthesis and Transformation for Autonomy (VISTA) [46] is a data-driven simulator freshly published by MIT. In contrast to previously presented softwares that are model based and highly engineered to simulate driving environments, VISTA mainly uses a deep neural network and a projection algorithm from 2D to 3D and vice versa to synthesize novel viewpoints (rotations up to $\pm 15^\circ$, translations up to ± 1.5 m, etc.) of a scene. The engine takes records from human driving scenarios and creates new photorealistic frames that are consistent with the original road environment. Vista provides several benefits over conventional simulators. For instance, the

TABLE III
COMPARISON OF END-TO-END DEEP-IMITATIVE MODELS FOR AUTONOMOUS DRIVING

Papers	Model architecture	Input(s)	Output(s)	Metric(s)	Result(s)	Dataset(s)	Year
Bojarski <i>et al.</i> [47]	CNN	YUV frame	Inverse turning radius	Autonomy	98%	Real world test	2016
Chen et Hung [49]	CNN	RGB frame	Steering angle	MAE	2.42°	Portion of Comma.ai	2017
Rausch <i>et al.</i> [50]	CNN	RGB frame	Steering angle	MEE	6.084°	CARSIM	2017
Jhung et al. [51]	CNN	RGB frame	Steering angle	N/A	N/A	Prescan Real world test	2018
Caltagirone <i>et al.</i> [52]	CNN	LiDAR Driver intention	Driving path	MaxF score	92.60%	KITTI	2017
Maqueda <i>et al.</i> [55]	CNN	Events frames	Steering angle	RMSE EVA	4.10° 0.805	DDD17	2018
Toromanoff <i>et al.</i> [53]	Ensemble learning with multiples CNNs	Frame from a fisheye camera	Steering angle	Autonomy #Laps achieved	95.23% +50	Own dataset Real world track	2018
Hubschneider <i>et al.</i> [54]	CNN	3 RGB frames	Steering angle	RMSE	15.81°	Private dataset	2017
Eraqi <i>et al.</i> [59]	CNN+LSTM	RGB frames	Steering angle	RMSE	14.93°	Comma.ai	2017
Xu <i>et al.</i> [30]	Dilated FCN + LSTM	RGB frames Speed Angular velocity	High level commands (straight, stop, left-turn, right-turn)	Perplexity accuracy	0.687 72.4%	BDDV	2017
Song <i>et al.</i> [61]	VGG-16 + LSTM	RGB frames	Steering angle	RMSE	2.44 °	ETS 2	2018
Fernando <i>et al.</i> [64]	CNN + LSTM	Grayscale frame Previous steers	Steering angle	RMSE	20.1° 22.5°	Comma.ai Udacity	2017
Yu <i>et al.</i> [32]	CNN	RGB frame	Vehicule curvature	MSE	1.24 · 10 ⁻⁶	BDD17	2017
Yu <i>et al.</i> [32]	CNN + LSTM	5 RGB frames	Speed	MSE	0.11	BDD17	2017
Yang <i>et al.</i> [68]	CNN + LSTM	RGB image Previous speeds	Steering angle Speed	MAE	1.26° 0.19 m/s	Udacity	2018
Chowdhuri et al. [68]	CNN	4 RGB images	Steering angle Speed	Autonomy	90%	Private dataset	2019
Codevilla <i>et al.</i> [69]	CNN	RGB frame Driver intention Speed Position...	Steering angle Acceleration	Successful episodes	77%	Carla	2018
Codevilla <i>et al.</i> [70]	CNN	RGB frame Driver intention Speed Position...	Steering angle Acceleration Speed	Successful episodes	92%	Carla	2019
Amini <i>et al.</i> [71]	VAE with CNN	RGB image	Steering angle Latent representation of the environment	MAE	4.4°	Private dataset	2018
Wenzel <i>et al.</i> [73]	Auto-Encoder + CycleGAN	RGB image	Steering angle	Successful turns	96%	Carla	2018

generation of new environments with varying challenging scenarios (eg. near-crash and off-road orientation) becomes easier, faster and safer without hurting the visual quality of the images. In addition, according to the authors, learned policies with VISTA are two times more likely to achieve better performances when transferred into real world roads compared to those trained on standard datasets or model based simulators. The reason is the fact that Vista enforces more robustness by making it easy to cover a wide range of complex scenarios and dangerous edge cases during learning.

IV. END-TO-END DEEP IMITATIVE MODELS

In this section, we will present recently proposed end-to-end methods using deep learning for lane keeping. A summary of the discussed techniques is provided in Table III. We divided this section into 5 parts: A. Pure imitative methods that make use of vanilla Convolutional Neural Networks (CNNs) and take standard camera frames only as input, B. Models that use other types of perceptual sensors such as events or fisheye cameras etc, C. Methods that consider previous driving history to estimate future driving commands, D. Systems that predicts both lateral

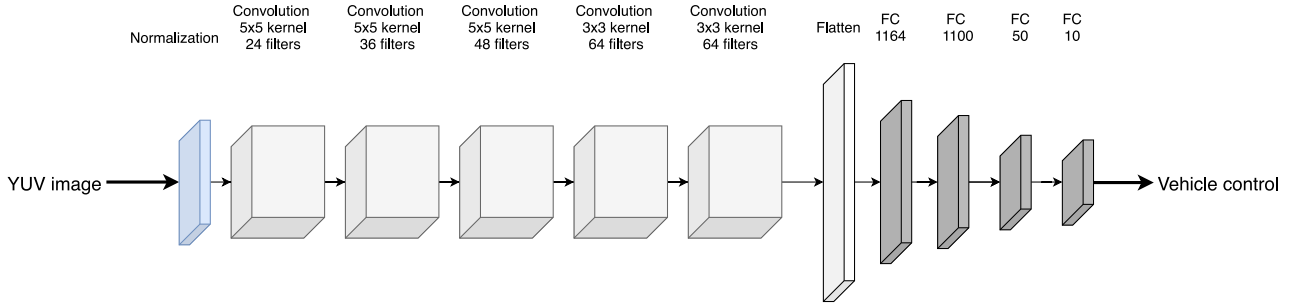


Fig. 2. CNN network architecture proposed by Bojarsky *et al.* [47]. The network starts with a normalization layer from which the output is fed to 5 convolutional layers with 5×5 and 3×3 kernels to produce a feature map that pass through 3 FC layers responsible for predicting control commands.

and longitudinal control commands, and finally E. Techniques that leverage the power of mid-level representations for transfer learning or give more explanation in regards to taken actions.

A. Using Natural Images as Input

Inspired by ALVINN [3], DAVE [4], and the recent advances in artificial neural networks, Bojarski *et al.* [47] developed DAVE-2 (renamed PilotNet [48]) an end-to-end deep learning approach for continuous car steering using a CNN. Their proposed network (see Fig. 2) is built in the following way: One normalization layer followed by 5 convolution layers serving as feature extractors or perception module, and finally three stacked Fully Connected (FC) layers which output the inverse turning radius ($1/radius$). Before feeding the image to the network, a conversion to YUV color space is applied. Additionally, most of the straight driving scenarios were removed from the training set to prevent bias. The loss is computed using the Mean Squared Error (MSE) between predictions and steering labels. Data used for training consist of videos recorded from three different front-facing cameras (left, center, right) placed behind the windshield of the car. Visual data are accompanied with measurements of the steering wheel angle. The recordings total 72 hours and include various weather conditions (clear, cloudy, foggy, rainy, snowy), times (day and night), and road types (with and without lane marking). The inverse turning radius is defined as the network output instead of the steering angle to make the model invariant to car geometry. Recovery from mistakes is made possible by adding synthesized data during training via simulations of car deviation from the center of the lane. Data augmentation was performed using a basic viewpoint transformation with the help of the left and right cameras. Visualization of some feature maps has shown that deep networks are able to implicitly infer from raw pixels perception sub-tasks such as lane markings, which would be learned individually in a modular pipeline approach. To evaluate their model, the authors defined the level of autonomy as follows:

$$\text{autonomy} = \left(\frac{E_t - N \cdot I_t}{E_t} \right) \cdot 100 \quad (1)$$

where E_t represents the total elapsed time, I_t the mean duration of an intervention fixed at 6 seconds and N being the number of interventions.

PilotNet performed very well on real world tests with approximately 98% of autonomy achieved, reviving the interest in behavior cloning approaches for autonomous driving.

Chen and Hung [49] proposed a simpler model consisting of 3 convolutional layers followed by a FC layer. Dropout is applied before and after the FC to help the network generalize better. The authors choice for a tiny CNN is driven by the small size of the training data picked from the Comma.ai dataset. Additionally, the model were trained and tested only on a portion of the dataset with clear time conditions. Thus, excluding night sequences and limiting the possibility of comparison with other deep models using the same dataset. The model was able to perform a Mean Absolute Error (MAE) of 2.42° on a test set corresponding to 2 video sequences totaling 25 K of daytime frames.

Similarly, Rausch *et al.* [50] and Jhung *et al.* [51] used the same network architecture as in [49] with just few differences such as applying dropout after every FC layer or adding more batch normalization layers. In [50], the model trained and tested on synthetic data from CARSIM simulator performed a minimum Mean Euclidean Error (MEE) of 6.084° . In [51], the authors used a private dataset recorded from Prescan (a popular commercial simulator for intelligent vehicles) to develop their model. A live test on a real world setting was successfully achieved.

B. Beyond Standard Frames as Input

While most of the approaches found in the literature make use of frames from traditional cameras as visual input, some recent work introduced other type of sensors for perception.

Caltagirone *et al.* [52] predicted driving path using raw data from sensors such as LiDAR and IMU, and the driving intention which is relevant in case where multiple drivable paths are available. GPS data coupled with information from IMU sensors and Google Maps are used to determine the driver intention. The Data consisted of 45 driving sequences totaling 4 days picked from the KITTI benchmarking suite. The Dataset includes different environment settings such as urban, highway and rural. After pre-processing, those data are fed to a Fully Convolutional Network (FCN) (see Fig. 3) which generates the pixels representing the drivable area of the input image, thus casting the task into a semantic segmentation problem. The FCN architecture consists of two parts: a downsampling stage

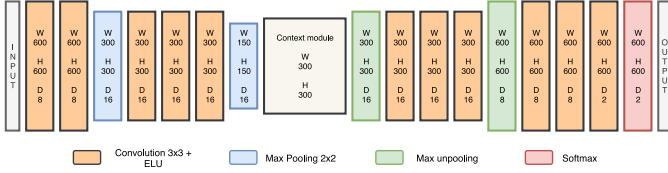


Fig. 3. Fully Convolutional Network (FCN) architecture proposed by Calta-girone *et al.* [52]. It takes LiDAR 3D point clouds as inputs and outputs high level actuation commands in the form of drivable path.

followed by upsampling giving a balance between performance and efficiency. Every stage in the network consists of five 3×3 convolutional layers. While two max pooling layers are naturally used during downsampling for dimensionality reduction, the upsampling stage makes use of 2 unpooling layers to restore the original input size.

Toromanoff *et al.* [53] used a single fisheye camera which is capable of capturing images with a wide angle for collecting visual data. With such type of sensor, they recorded 1000 km or 200 hours of driving including highway, urban, country road driving and various weather conditions in the Paris region. The dataset also includes challenging obstacle avoidance and sharp turns. A more realistic label augmentation is achieved with the help of the wide range of captures from the front fisheye camera compared to previous methods using shearing with side (right and left) cameras (Hubschneider *et al.* [54], Bojarski *et al.* [47]). The authors used a built-in simulator as evaluation metric and argued that it is more reliable for real world tests than error functions such as MSE which do not take into consideration accumulated errors that can lead to a deviation from the lane. In that, their proposed evaluation tool can provide more accurate results on the number of unacceptable deviations in an offline environment. The same network architecture as in [47] was used with the only notable difference being the addition of the L2 regularization term in the loss function (expressed in Equation 2) for better generalization.

$$L(Y, \bar{Y}, \theta) = L_{MSE}(Y, \bar{Y}) + \lambda_{reg} L_{2reg}(\theta) \quad (2)$$

where Y and $\bar{Y}$ are the ground truth and predicted angles, respectively, θ the network parameters, and λ a constant fixed at $5 \cdot 10^4$. Note that, it is not the raw images from the fisheye cameras that are directly fed to the network but results from projections into a cylinder perpendicular to the ground plane. This is to remedy for the distortion caused by the wide range of the captures. The final model was a result of an ensemble learning method called bagging where multiple trainings are executed each on a subset of the dataset. 99.5%, 98.7% and 87.5% of autonomy were achieved respectively for urban, highways and sharp turns scenarios from their custom dataset. Regarding real world performance, their system was able to drive without any human intervention for more than 50 laps on a test track. The authors also shared a video of their trained agent driving autonomously on Grand Theft Auto for about 1mn and 45 s in a country road setting, this was achieved without ever being trained on images from the video game.

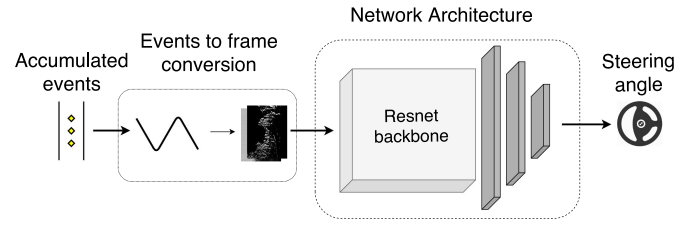


Fig. 4. Summary of the approach proposed by Maqueda *et al.* [55]. Sequences of events are converted into the same format as standard frames, which are then fed to a fine-tuned Resnet to output steering wheel angle.

Following the same idea of using different visual sensors, Maqueda *et al.* [55] made use of data from a front-facing event-based camera instead of a standard camera to output a steering angle prediction. Indeed, events based cameras consist of independent pixels that record intensity variation in an asynchronous way. Thus, giving more information in a time interval than traditional video cameras where changes taking place between two consecutive frames are not captured. However, the challenge faced by the authors here is that existing deep convolutional architectures are not designed to deal with events but rather traditional frames as input. To tackle this issue, the authors converted the generated events to conventional frames format. Their approach consisted of accumulating events over a constant time interval T and producing a frame (similar to those produced by normal cameras). To dig more into the details, let's consider an event represented by $e^k = (x^k, y^k, t^k, p^k)$ where x^k and y^k are the spatial coordinates of the receptive pixel, t^k is the point in time at which the event is captured, and p^k is the event polarity or sign of the intensity change (p^k is equal to -1 if the polarity is negative or $+1$ otherwise). In every time interval, a negative and positive histograms are constructed from accumulated negative and positive events, respectively, using the following mathematical formula:

$$h^{\pm}(x, y) = \sum_{t_k \in T, p_k = \pm 1} \delta(x - x_k, y - y_k), \quad (3)$$

where δ is the Kronecker delta. The number of bins in each histogram correspond to the number of individual pixels with coordinates x and y . After that stage, the two histograms are stacked to form an event-based frame. Those frames are used as inputs of the proposed model with a ResNet [56] backbone, pretrained on ImageNet [57] dataset, as a feature extractor. Two variants of ResNet (ResNet18 and ResNet50) were considered as visual encoders by the authors. Then, each of them is topped with a 2D global average pooling layer before plugging a 256-dimensional and 1024-dimensional FC layer for ResNet18 and ResNet50, respectively. Finally, a rectified linear unit (ReLU) [58] activation layer followed by a one dimensional FC layer are joined to each variant to output the steering angle. The overall approach is summarized in Fig. 4. Training and evaluation were performed using DDD17 dataset [34]. The results show state of the art performance with 0.826 and 4.10° on EVA (Explained Variance) and RMSE (Root Mean Squared Error) metrics, respectively, for the ResNet50 based model.

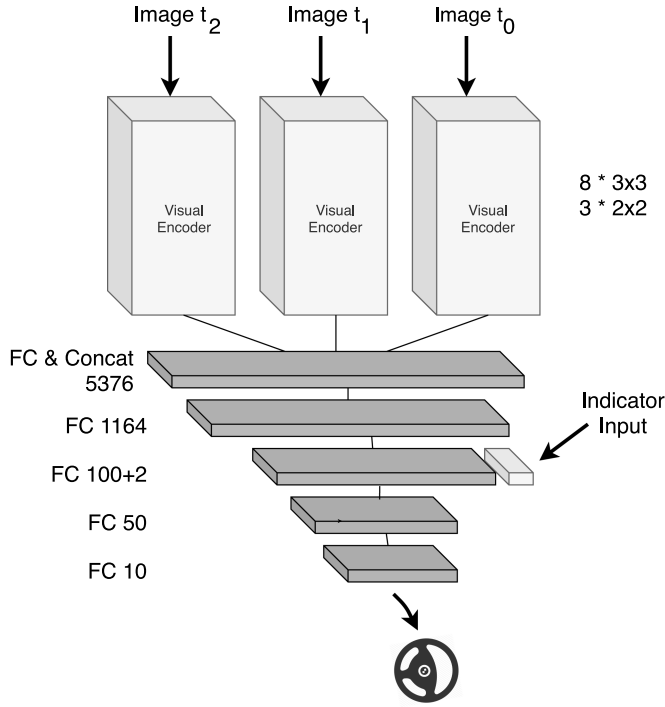


Fig. 5. DriveNet network architecture [54]. The network takes 3 consecutive images spaced by 10 m each and a driving intention to predict a steering angle.

C. Considering Spatio-Temporal History

Models presented above make use of a single image captured from a front facing road camera to infer vehicle control. But in order to achieve better performance, many recent works integrated spatio-temporal information by considering a sequence of images as inputs.

Hubschneider *et al.* [54] developed DriveNet, a model that takes 3 images separated by 10 m of driving to capture a wider angle of the environment from a single camera. The authors argued that this technique is useful for driving in urban areas which usually require sharp turns. DriveNet also integrates turning decision (go straight, turn left, and turn right) into the driving process, while most of the previous methods focused on lane following and would require manual control when a change of direction is needed. DriveNet, as illustrated in Fig. 5, is structured as follows: three similar visual encoders with each handling one of the input images of size 254×52 and consisting of a stack of eight 3×3 convolutional layers and 2×2 max pooling, then the feature maps generated from those networks are flattened and concatenated before feeding them to five consecutive FC layers. The turning signal is introduced only at the second FC layer. Uniform distance distribution of 10 m is chosen over a time period distribution of frames by the fact that the latter is highly dependant on velocity and thus may fail to capture enough information when the car moves very slowly. Additionally, as in [47], the authors of DriveNet made use of data augmentation to simulate deviation from the center of the lane in order to allow their system to recover from errors. The transformations include image shearing and off-center crop

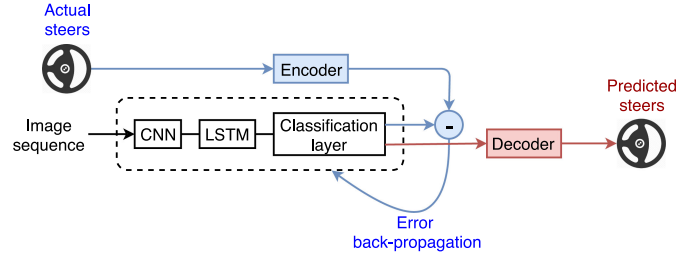


Fig. 6. C-LSTM network architecture proposed in [59] to integrate temporal history for steering prediction. The blue and red path correspond to training and testing phases respectively. During training, measured steering wheel angles are converted into discrete values using an encoder following a sine wave function. Hence during deployment, the predicted classes are decoded to give a smoother angle.

to simulate shifts and rotations, respectively. An evaluation on a sequence of 400 images recorded by the authors shows an RMSE error of 15.81° . Finally, a live test on 11.8 km long highway segment demonstrated a very good performance of DriveNet in a real world scenario since the authors reported that no human intervention was required at all.

Eraqi *et al.* [59] proposed a Convolutional Long Short Term Memory [60] (C-LSTM) architecture (Fig. 6) to integrate temporal dependencies for a more efficient steering angle prediction from a single front-facing camera. The CNN is used as an encoder for visual data and processes one frame at a time. The resulting features of length 2048 are passed to the LSTM units responsible for introducing 5 seconds of driving history before computing the intermediate output with a classification layer activated by a tanh function. In contrary to the previous methods, the authors transformed the problem of steering angle prediction into a logistic regression task by encoding the ground-truths with a specific sine wave function during training. Hence, predictions are decoded during the deployment phase to produce the desired angle in degree. The sine wave function is given by

$$Y_i = \sin \left(\frac{2\pi(i-1)}{N-1} - \frac{\phi\pi}{2\phi_{\max}} \right), \quad (4)$$

with N being the number of the classification layer neurons, ϕ the angle to encode, Y_i corresponding to the activation of the output neuron i , N and $\phi_{\max}$ are fixed to 95 and 190° , respectively. The motivation behind this choice is that it enables the output layer neurons to have a spatial relationship. The results of their experiments reported using the RMSE metric have shown that their C-LSTM classifier model gives a better performance with an average error of 14.93° compared to 17.77° and 23.30° for ResNet and PilotNet respectively. Also, a comparison against a version of their proposed C-LSTM model predicting directly continuous values (as commonly seen in the literature) instead of discrete values has shown that defining the problem as a classifier can help increase performance. In this particular case, the error was reduced by 1.08° . The Comma.ai dataset was used for benchmarking.

Xu *et al.* [30] proposed a generic driving model trained from large scale uncalibrated video data. The model is generic in that it predicts future motion path or feasible actions (for instance:

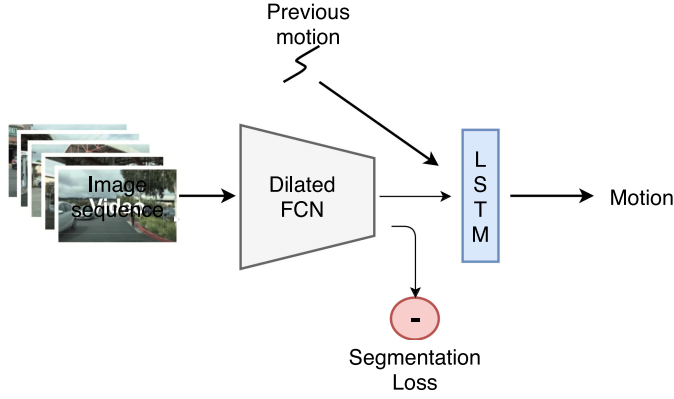


Fig. 7. FCN-LSTM network architecture proposed by Xu *et al.* [30]. Features maps of consecutive RGB images generated with a dilated FCN and the previous motion are fed to a LSTM module to predict motion in the form of a sequence of high level control actions.

straight, stop, left-turn, right-turn) given by a sequence of commands to execute. Their network architecture (Fig. 7) consists of a FCN-LSTM network that combines two losses: demonstration and segmentation loss. This approach of aggregating both losses is called privileged training and could considerably speed up the learning process according to the authors. The visual encoder used to extract perceptual representations consists of a pretrained AlexNet, modified to integrate dilated convolutions. The visual encoder is then jointly fine tuned with a 64 units LSTM layer. This second stage is responsible for fusing previous states and the current states making the predictions more robust. Besides visual sequences, states comprise previous motion represented by car speed and angular velocity. To evaluate their work, the authors defined perplexity, a metric suitable for models where sequences of words are predicted. The perplexity is expressed by:

$$\text{perplexity} = \exp \left\{ -\frac{1}{t} \sum_{i=1}^t \log F(s_i, a_i) \right\} \quad (5)$$

where s_i and a_i represent the state and action at timestep i , $F(s, a)$ measures the feasibility of action a under state s , and t is the number of time steps. The training and evaluation of their model is done on a sample comprising 26,839 videos from their jointly proposed dataset BDDV with each sequence lasting approximately 40 seconds. The data are approximately split as follows: 81.25% for training, 5.5% for validation, and 13.25% for testing. The images are re-sized to 640 x 360 pixels and the frame rate of the videos are reduced to 3 Hz to prevent redundancy in visual inputs. Finally, the authors reported a performance of 0.697 perplexity and 72.4% accuracy for the privileged training approach compared to 0.718 / 71.31% and 0.8887 / 61.66% for motion reflex and mediated perception approaches, respectively.

Song *et al.* [61] used a similar architecture to Xu *et al.* [30], except that the perceptual encoder is a VGG-16 [62] instead of a Dilated FCN [63]. The network takes 11 consecutive images as input and outputs a steering angle prediction. Stochastic gradient descent optimizer and a MSE loss were used for training the

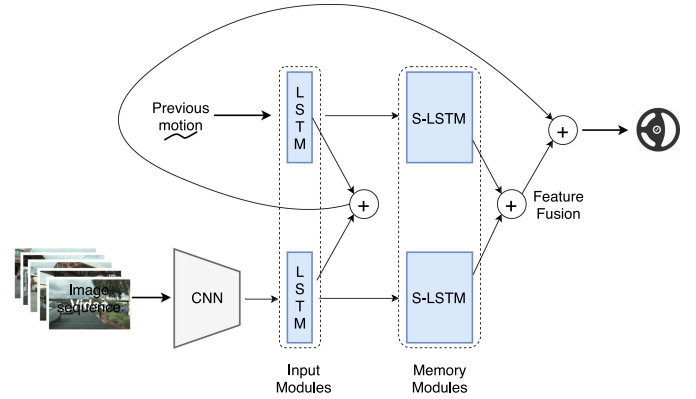


Fig. 8. Method proposed by Fernando *et al.* [65] using a tree network architecture with multiple LSTM modules to capture both short and long term dependencies.

model. Moreover, data were collected from ETS 2 simulator for both training and evaluation. Reported results show a RMSE of approximately 1.72°, 2.97°, and 2.64° for country road, mountain road, and freeway, respectively.

Fernando *et al.* [64] proposed a method based on Tree Memory Networks [65] (Fig. 8) which has shown greater performance for path planning compared to sequential memory networks to capture not only short term dependencies (as in the model proposed in [30]) but far earlier historical dependencies such as traffic conditions, weather conditions, etc. As in [30], their model takes visual and motion inputs but there is one input module specific for each of the spatial and trajectory inputs. The trajectory in this approach is simply represented with steering wheel angle and not with car speed or angular velocity as found in [30]. On top of that, each module is extended with a second node called memory modules to capture different features of temporal dependencies. After that stage, spatial and trajectory features are fused and concatenated with the outputs of the input modules to generate a control command. Various fusion methods were tested and the best performing is an hierarchical fusion approach that consists of combining the different outputs of the four modules with a recursive LSTM [60]. The authors adopted a pre-trained VGG-16 [62] backbone to extract visual meaningful features to fed to the visual input module. For experiments, the authors worked with two datasets: Comma.ai and Udacity self driving car datasets to evaluate and compare their model against other state of the art models. The results show that their model performs better than some previously proposed methods with a RMSE of 20.1° for Comma.ai and 22.5° for Udacity datasets compared to 23.1 and 27.3°, 25.6° and 31.8° for models in [30] and [66], respectively.

D. Adding Longitudinal Control to the Equation

While the previously described papers focus only on predicting lateral control such as steering angle, drivable path, etc., Yu *et al.* [32] introduced the estimation of longitudinal control to the equation. They made use of two separate networks with each focusing on predicting one sub-task. The first network is

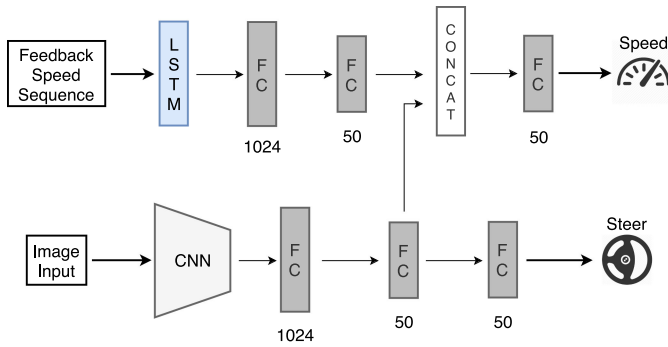


Fig. 9. Multi-Modal multi-task network proposed by Yang *et al.* [67] to predict both steering angle and speed at once. Steering angle is estimated from a single input image but speed is predicted by taking into account previous motions in the form of a sequence of speed as well.

very similar to PilotNet with 5 convolutional and 2 FC layers and outputs the vehicle curvature instead of the steering angle as generally found in the literature, which is justified by the fact that curvature is more general and does not vary from vehicle to vehicle. The second network responsible for predicting the longitudinal control makes use of a CNN-LSTM architecture. While the authors considered a single image from a front-facing camera might be sufficient for lateral control, they concluded that many frames are necessary to judge whether other traffic participants are moving or not. If yes, then at which speed? In that, the input is defined as a spatio-temporal sequence justifying the use of LSTM units. The structure of their second network consists of 5 consecutive Convolutional-LSTM modules followed by one FC layer. Their model takes 5 frames at 80 x 80 resolution as input, each captured at a rate of 8 fps from a front facing camera. In their reported results, the Baidu researchers were able to reach an error of approximately 10^{-6} in the curvature estimation task and 0.11 in the longitudinal control prediction task using BDD17 dataset released publicly with their paper.

Instead of having a separate network for each task, Yang *et al.* [67] proposed a multi-modal multi-task vehicle control network (see Fig. 9) that predicts both steering angle and vehicle speed instead of having two distinct networks. Their model takes two inputs: a single image from a front-facing camera and previous vehicle speed. As in [32], the steering angle output depends solely on the visual input but the speed is predicted by using also previous vehicle motions instead of just using a sequence of frames. The visual encoder comprises 5 convolutional layers with decreasing receptive fields as the network gets deeper. Normalization and pooling are applied on the outputs of the first two convolutions. In the one hand, generated feature maps are passed through three FC layers with respectively 1024, 50, and 50 neurons to output steering angle. In the other hand, current vehicle speed is fed to a LSTM layer with 128 latent dimensions followed by two FC layers with 50 neurons representing past vehicle motions. The encoded past motions are concatenated with the visual features maps and passed through a FC layer of 50 neurons to predict future vehicle speed. Experiments are done using a dataset extracted from the popular Udacity simulator and a home made dataset containing 5 hours of driving data collected

in San Jose (CA, US). The results are reported using the MAE metric. The authors claimed their model was able to reach an error of only 1.26° with Udacity compared to 4.26° for Nvidia PilotNet [47] on the steering angle prediction task. Finally, 0.19 and 0.45 m/s error rate were reported for Udacity and their private dataset, respectively, on the longitudinal prediction task.

Similarly to Yang *et al.* [67], Chowdhuri *et al.* [68] proposed a multi-modal multi-task control network named MultiNet for off-road driving. However, instead of considering previous speeds as second inputs, the model takes a driving mode (among direct, follow and furtive) along with 4 RGB images to predict a steering angle and a value for speed. Moreover, the proposed network does not have a recurrent layer in contrast to the multi-task network in [67]. With carried out experiments using 1/10th scaled vehicles in unstructured environments, the model achieved 92.68% and 88.23% of autonomy in direct and furtive mode driving, respectively.

Codevilla *et al.* [69] proposed a multi-modal multi-task approach conditioned by the driver intention to predict both steering angle and acceleration. The authors used the driver intention (navigation command) as an auxiliary input during training instead of inferring the action solely from the visual input to deal with some ambiguity that can occur when the vehicle is close to an intersection for instance. In that, the authors proposed two different network architectures (Fig. 10): a first network in which the navigation command is processed as input and another network where the latent variable representing the driver intention is used as a switch that activates a specific sub-module among three. Data augmentation techniques similar to [48] are applied to simulate shifts and rotations. On top of that, a noise is injected into the experts control signal making the models more robust by improving their capability to recover from drifts. Training and testing were done using Carla in two different towns and the models are evaluated using the percentage of successful episodes, defined as episodes where the agent were able to reach the final destination. The second proposed version performs dramatically better with 88% and 64% success rate respectively in Town 1 and 2 compared to just 20% and 26% achieved by the baseline non-conditional method. Further experiments were made in real world environment with a 1/5th scaled vehicle in a route consisting of 14 intersections resulting in the branched model performing 100% of turns successfully across 3 runs. In a following work [70], the authors modified the architecture of their network to use a Resnet perception module and also output speed in addition to steering angle and acceleration. With this new approach, an average improvement of 3.5% successful episodes was observed in the original Carla Benchmark.

E. Exploiting Intermediate Representations

Amini *et al.* [71] used a variational autoencoder (VAE) model to predict steering angle as well as extract latent representation from the visual inputs to respond to questions such as: what are the weather conditions? what is the situation of the road? is there a vehicle ahead etc?. Unlike end-to-end models that just output an actuation value, representing information about the environment of the vehicle can give more explanation on what

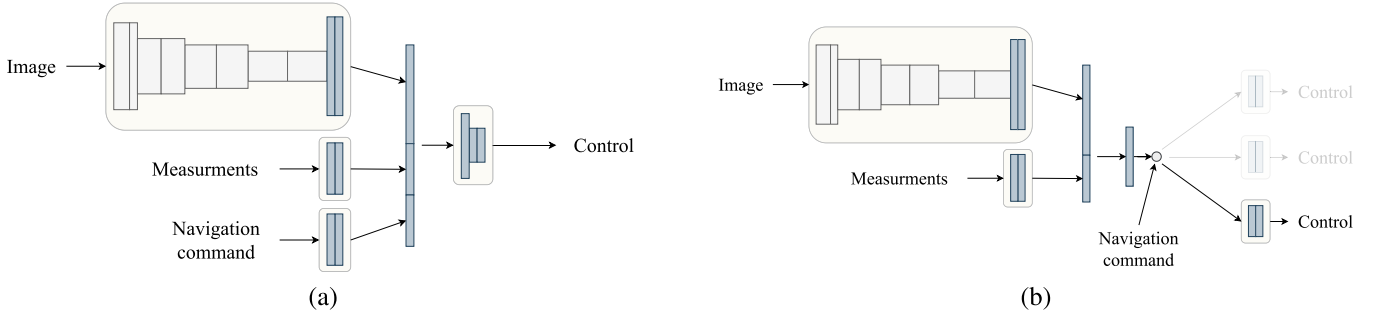


Fig. 10. Two network architectures proposed by Codevialla *et al.* [69]. On Top of the sensor data, the models are also fed with the driver intention or navigation command. In (a) the navigation command is considered as a simple input while in (b) it acts like a switch.

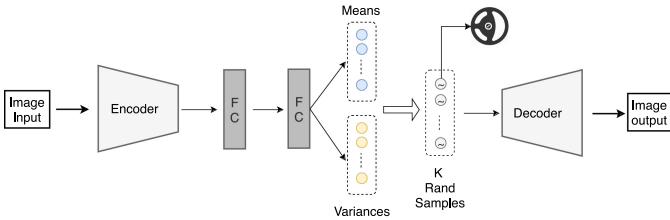


Fig. 11. VAE network architecture proposed by Amini *et al.* [71]. The model not only predict control command, but also attempt to represent environment states with 25 latent variables giving more information on why decisions are made. A subset of those latent variables is used for predicting steering angle while the whole helps in reconstructing the input during training.

the deep network is learning and why some decisions are made. Regarding the network architecture and learning parameters, one image of size $60 \times 200 \times 3$ is given as input and training is done with a mini-batch of size 50. The encoder comprises 5 convolutional layers with filters of size 5 and 3 activated with ReLU non-linearity function. This feature extractor block is followed by 3 FC layers which output 2 k values representing the latent space with k being the size of the sampled vector fed to the decoder. Dropout is applied for better generalization. The whole sample from the latent space is used to regenerate the input image but only a portion of it is utilized to predict the steering angle. The decoder stage consists of the same layers found in the encoder part but appear in the inverse order. Indeed, convolutions are replaced with deconvolutions to upscale the input until the initial size of the image is reached.

Three losses including the reconstruction loss, the latent loss and the prediction loss (also called supervised latent loss by the authors) are combined to train the VAE. The reconstruction loss or L_1 loss is the MAE between the input (x) and the output ($\hat{x}$) frames. The distance between latent vectors and a unit Gaussian with a mean μ and standard deviation σ represent the latent loss and is computed using the Kullback-Leibler divergence (KL) [72]. Finally, the prediction loss or L_2 norm is the MSE between the observed (y) and the predicted ($\hat{y}$) values for the curvature. Each loss function is multiplied by a constant to weight its importance during training. The authors found that the best values for these constants are $c_1 = 0.1$, $c_2 = 0.033$, and $c_3 = 0.001$ respectively for the prediction loss, the

reconstruction loss, and the latent loss. The combined loss function is given by the following equation:

$$L_{Total} = c_1 L_1(x, \hat{x}) + c_2 L_2(y, \hat{y}) + c_3 L_{KL}(\mu, \sigma) \quad (6)$$

Additionally, the motivation behind using a VAE architecture is to automatically mitigate the bias issue which occurs because generally the driving scenes in the datasets does not have the same proportions. In previous methods, this issue is solved by manually reducing the over represented scenes such as straight driving or stops. In their approach, Amini *et al.* approximates the distribution of the training data following both the inputs (for instance some weather conditions may be under-represented) and the outputs (for example straight driving command are likely to be more common) in the latent space during a first phase of training that involves the whole dataset. Then, training with a subsample of the dataset almost uniformly distributed over the latent variables will lead to the model performing better for less frequent examples while just slightly decreasing the performance for usual scenarios. For data collection, a Toyota Prius 2015 V equipped with a wide angle (120°) facing road camera capable of capturing high definition frames at ~ 120 Hz is driven in the Boston Metropolitan area for a total of 4 hours. Along the RGB images, data such as steering wheel angle, speed, rotation and orientation of the car were also collected from different sensors including IMU. A NVIDIA PX2 is used for computation and integrating data coming from different sources. As in [47], data is augmented using two side cameras to enable recovery capability when the vehicle deviates. Training is done using Adam optimizer with a learning rate of $\alpha = 0.0001$, $\beta_1 = 0.9$, and $\beta_2 = 0.999$. Different number of latent variables were tested and $k = 25$ gave the best overall results. To evaluate their VAE model, the authors compared it with a simple regressor similar to the encoder part but with only one output being the steering angle. The proposed VAE model performance is slightly worse with an error of 4.4 compared to 3.8 for the second model. But, it should be noted that the VAE model comes with additional benefits such as automatic debiasing and latent representation of the environment capabilities which are supported by experimental tests.

Following the same line of reasoning to leverage mid-level representation, Wenzel *et al.* [73] took advantage of recent advances in image translation allowed by Generative

Adversarial Networks (GANs) [74] to transfer visual data from one weather to another and generalize their driving model for various meteorological conditions. The proposed framework consists of three units. A perception module which makes use of an encoder-decoder architecture that generates, from raw inputs, intermediate representations in the form of semantic maps. A controller module which takes the rendered images and outputs steering command. A transfer module consisting of a cycleGAN network [75] to translate in an unsupervised manner input images from a different weather condition to something similar to what the perception module was trained on. For the sake of improving generalization performance on unseen domain, a second perception module is trained in pair with the first one to produce latent vectors similar to those generated by the latter but by taking translated images as input. To sum up, the training process consisted of feeding the first perception module with original images and the second perception module with translated frames, the loss is computed with a MSE between the generated latent vectors from these two networks. This way, the model is more robust because the second perception model is able to produce latent vectors close to what the decoder expects. Hence, correct segmented images can be generated and fed to the controller network. Evaluation focused on curved roads using Carla simulator which has 14 various weather conditions has shown that their model when tested with unknown domain were able to successfully complete an average of 96% of turns.

V. CHALLENGES AND OPPORTUNITIES

Recent years have seen a tremendous amount of papers in the field of autonomous driving that make use of supervised deep learning models, thanks to the availability of more real world datasets and simulation softwares especially with the increasing involvement of technology companies. Models making use of non-standard cameras or intermediate representations are showing a lot of potential in comparison to pure imitative methods that takes conventional video frames as input. However, as noticed in Section IV, the diversity in metrics and datasets used for reporting the results makes it very hard to strictly weigh the different models against each other and draw the right conclusions regarding their performance. A good start toward solving this issue would be to have a benchmark of the various proposed models on multiple available datasets and simulators. Besides that, there is room for improvement in many other realms.

More data: Data is crucial for research especially in the context of deep learning. This is even more important for lane keeping or autonomous driving in general where the scenarios are extremely diverse and it is difficult to cover all of them. There are many parameters to take into account. For instance the weather conditions can range from hot to cold, wet to dry, calm to stormy, clear to cloudy, etc. The time can range from day to night including sunrise and sunset. The environment can have different states including city, town, country-side to name just a few. Hence, there is a lot of work to be done in this area despite all efforts made until now with increasing real world datasets and simulators being released to the public, especially in the recent years.

More emphasis in non standard visual data: Most of the methods found in the literature make use of frames from conventional video cameras. This is mainly due to their wide availability and relatively low cost compared to other perceptual sensors such as LiDAR. But, for instance, Maqueda *et al.* [55] has shown that non standard visual data can give better performance compared to RGB images. As such, we suggest further exploration of other types of perceptual sensors when collecting new data, and developing simulators and driving models.

Explainability and transparency of taken decisions: Over the years, proposed end-to-end frameworks has shown great potential by continuously decreasing the error rate in predicting vehicle commands. However, one of the most challenging task with end-to-end deep driving models remains explainability in decision making despite this being an open area of research in deep learning. This situation makes it extremely hard to trust such approaches in real world roads where unpredictable actions are not acceptable given the high risks involved in autonomous driving. We believe more initiatives need to be directed toward this broad issue when designing driving models. A common approach in the literature is to analyse the pixels that lead to the greatest activation of neurons. Such technique is used in [48] to determine how deep networks automatically derive the necessary domain knowledge (cars, lane markings...) from human examples. Similarly, leveraging the power of intermediate representations through VAE architectures, as shown in [71], may provide some insights in what the network sees. Furthermore, for more transparency, it would be interesting to have end-to-end models that predict some elements of the environment (lane marking, distance of the vehicle ahead, etc.) in parallel to actuation commands.

VI. CONCLUSION

Autonomous driving is a broad and challenging task to solve. With the advent of deep learning in the last decade, several approaches based on mimicking human behavior for vehicle control have been proposed recently. In this paper, we provided an overview of the current state of the art deep behavior cloning methods for lane stable driving. We also presented some publicly available datasets and simulators for the development of such models. Furthermore, we described some of the current challenges and upcoming opportunities for end-to-end frameworks in the context of self-driving vehicles. With this work, we hope to give new researchers a first insight on the recent development in the field, as well as help more experienced researchers keep pace with the rapid advances in the domain.

ACKNOWLEDGMENT

The authors acknowledge the support of the Natural Sciences and Engineering Research Council of Canada (NSERC), [funding reference number RGPIN-2018-06233].

REFERENCES

- [1] T. Litman, "Autonomous vehicle implementation predictions," Victoria Transport Policy Institute, Victoria, BC, Canada, Tech. Rep., Oct. 2019. [Online]. Available: <https://www.vtpi.org/avip.pdf>

- [2] E. D. Dickmanns and V. Graefe, "Applications of dynamic monocular machine vision," *Mach. Vision Appl.*, vol. 1, pp. 241–261, 1988.
- [3] D. Pomerleau, "ALVINN: An autonomous land vehicle in a neural network," in *Proc. Neural Inf. Process. Syst.*, pp. 305–313, 1988.
- [4] Net-Scale Technologies, Inc., "Autonomous Off-Road Vehicle Control Using End-to-End Learning," DARPA-IPTO Arlington, VA & Net-Scale Technologies, Inc., Morganville, NJ, USA, Tech. Rep. ARPA Order No. Q458, Jul. 2004.
- [5] C. Urmson *et al.*, "Autonomous driving in traffic: Boss and the urban challenge," *AI Mag.*, vol. 30, pp. 17–28, 2009.
- [6] M. Montemerlo *et al.*, "Junior: The Stanford entry in the urban challenge," *J. Field Robot.*, vol. 25, pp. 569–597, 2008.
- [7] A. Bacha *et al.*, "Odin: Team Victortango's entry in the DARPA urban challenge," *J. Field Robot.*, vol. 25, pp. 467–492, 2008.
- [8] N. Groom, "Tesla unveils all-wheel drive Model S, 'autopilot' features," Oct. 2014. [Online]. Available: <https://www.reuters.com/article/us-tesla-motors-launch-idUSKCN0HZ07O20141010>
- [9] A. C. Madrigal, "Waymo's robots drove more miles than everyone else combined," Feb. 2019. [Online]. Available: <https://www.theatlantic.com/technology/archive/2019/02/the-latest-self-driving-car-statistics-from-california/582763/>
- [10] A. Best, S. Narang, D. Barber, and D. Manocha, "AutonoVi: Autonomous vehicle planning with dynamic maneuvers and traffic constraints," in *Proc. Int. Conf. Intell. Robots Syst.*, 2017, pp. 1161–11618.
- [11] R. Matthaei and M. Maurer, "Functional system architecture for an autonomous on-road motor vehicle," in *Automotive Systems Engineering II*. Berlin, Germany: Springer, 2018.
- [12] C. Chen, A. Seff, A. Kornhauser, and J. Xiao, "DeepDriving: Learning affordance for direct perception in autonomous driving," in *Proc. IEEE Int. Conf. Comput. Vision*, 2015, pp. 2722–2730.
- [13] A. Sauer, N. Savinov, and A. Geiger, "Conditional affordance learning for driving in urban environments," in *Proc. Conf. Robot Learn.*, 2018, pp. 237–252.
- [14] S. Aradi, T. Becsi, and P. Gspr, "Policy gradient based reinforcement learning approach for autonomous highway driving," in *Proc. IEEE Conf. Control Technol. Appl.*, 2018, pp. 670–675.
- [15] T. de Bruin, J. Kober, K. Tuyls, and R. Babuka, "Integrating state representation learning into deep reinforcement learning," *IEEE Robot. Autom. Lett.*, vol. 3, no. 3, pp. 1394–1401, Jul. 2018.
- [16] M. Jaritz, R. de Charette, M. Toromanoff, E. Perot, and F. Nashashibi, "End-to-end race driving with deep reinforcement learning," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2018, pp. 2070–2075.
- [17] M. Kaushik, V. Prasad, K. M. Krishna, and B. Ravindran, "Overtaking maneuvers in simulated highway driving using deep reinforcement learning," in *Proc. IEEE Intell. Veh. Symp.*, 2018, pp. 1885–1890.
- [18] G.-H. Liu, S. Avinash, S. Prabhakar, M. M. Veloso, and G. Kantor, "Learning end-to-end multimodal sensor policies for autonomous navigation," in *Proc. Conf. Robot. Learn.*, 2017, pp. 249–261.
- [19] L. Tai, J. Zhang, M. Liu, J. Boedecker, and W. Burgard, "A survey of deep network solutions for learning control in robotics: From reinforcement to imitation," 2016, *arXiv:1612.07139*.
- [20] B. Paden, M. Čáp, S. Z. Yong, D. Yershov, and E. Frazzoli, "A survey of motion planning and control techniques for self-driving urban vehicles," *IEEE Trans. Intell. Veh.*, vol. 1, no. 1, pp. 33–55, Mar. 2016.
- [21] C. Badue *et al.*, "Self-driving cars: A survey," 2019, *arXiv:1901.04407*.
- [22] M. E. Campbell, M. Egerstedt, J. P. How, and R. M. Murray, "Autonomous driving in urban environments: Approaches, lessons and challenges," *Philosoph. Trans. Series A, Math., Physical, Eng. Sci.*, vol. 368, no. 1928, pp. 4649–4672, 2010.
- [23] C. Katrakazas, M. Qudus, W.-H. Chen, and L. Deka, "Real-time motion planning methods for autonomous on-road driving: State-of-the-art and future research directions," *Transp. Res. Part C: Emerg. Technol.*, vol. 60, pp. 416–442, 2015.
- [24] J.-R. Xue, J.-W. Fang, and P. Zhang, "A survey of scene understanding by event reasoning in autonomous driving," *Int. J. Autom. Comput.*, vol. 15, no. 3, pp. 249–266, 2018.
- [25] Y. Kang, H. Yin, and C. Berger, "Test your self-driving algorithm: An overview of publicly available driving datasets and virtual testing environments," *IEEE Trans. Intell. Veh.*, vol. 4, no. 2, pp. 171–185, Jun. 2019.
- [26] A. Geiger, P. Lenz, and R. Urtasun, "Are we ready for autonomous driving? The KITTI vision benchmark suite," in *Proc. IEEE Conf. Comput. Vision Pattern Recognit.*, 2012, pp. 3354–3361.
- [27] F. Yu *et al.*, "BDD100k: A diverse driving video database with scalable annotation tooling," in *Proc. IEEE Conf. Comput. Vision Pattern Recognit.*, 2020.
- [28] E. Santana and G. Hotz, "Learning a driving simulator," 2016, *arXiv:1608.01230*.
- [29] H. Schafer, E. Santana, A. Haden, and R. Biasini, "A commute in data: The comma2k19 dataset," 2018, *arXiv:1812.05752*.
- [30] H. Xu, Y. Gao, F. Yu, and T. Darrell, "End-to-end learning of driving models from large-scale video datasets," in *Proc. IEEE Conf. Comput. Vision Pattern Recognit.*, 2017, pp. 3530–3538.
- [31] W. Maddern, G. Pascoe, C. Linegar, and P. Newman, "1 year, 1000 km: The Oxford Robotcar Dataset," *Int. J. Robot. Res.*, vol. 36, pp. 3–15, 2017.
- [32] H. Yu, S. Yang, W. Gu, and S. Zhang, "Baidu driving dataset and end-to-end reactive control model," in *Proc. IEEE Intell. Veh. Symp.*, 2017, pp. 341–346.
- [33] Y. Chen *et al.*, "LiDAR-video driving dataset: Learning driving policies effectively," in *Proc. IEEE Conf. Comput. Vision Pattern Recognit.*, 2018, pp. 5870–5878.
- [34] J. Binas, D. Neil, S.-C. Liu, and T. Delbruck, "DDD17: End-To-End DAVIS driving dataset," in *Proc. ICML Workshop Mach. Learn. Auton. Veh.*, 2017. [Online]. Available: <http://proceedings.mlr.press/>
- [35] A. Geiger, P. Lenz, C. Stiller, and R. Urtasun, "Vision meets robotics: The KITTI dataset," *Int. J. Robot. Res.*, vol. 32, no. 11, pp. 1231–1237, 2013.
- [36] B. Goldfain *et al.*, "AutoRally: An open platform for aggressive autonomous driving," *IEEE Control Syst. Mag.*, vol. 39, no. 1, pp. 26–55, Feb. 2019.
- [37] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, "CARLA: An open urban driving simulator," in *Proc. Conf. Robot Learn.*, 2017, pp. 1–16.
- [38] S. Shah, D. Dey, C. Lovett, and A. Kapoor, "AirSim: High-fidelity visual and physical simulation for autonomous vehicles," in *Field and Service Robotics*, Springer Proceedings in Advanced Robotics. Berlin, Germany: Springer, 2018, pp. 621–635.
- [39] A. Best, S. Narang, L. Pasqualin, D. Barber, and D. Manocha, "AutonoVi-Sim: Autonomous vehicle simulation platform with weather, sensing, and traffic control," in *Proc. IEEE Conf. Comput. Vision Pattern Recognit. Workshops*, 2018, pp. 1161–11618.
- [40] M. Müller, V. Casser, J. Lahoud, N. Smith, and B. Ghanem, "Sim4cv: A photo-realistic simulator for computer vision applications," *Int. J. Comput. Vision*, vol. 126, no. 9, pp. 902–919, 2018.
- [41] N. Koenig and A. Howard, "Design and use paradigms for Gazebo, an open-source multi-robot simulator," in *Proc. /RSJ Int. Conf. Intell. Robots Syst.*, 2004, pp. 2149–2154.
- [42] Udacity, "A self-driving car simulator built with Unity," 2017. [Online]. Available: <https://github.com/udacity/self-driving-car-sim>
- [43] A. Ruano, "aitorzip/DeepGTAV," 2016. [Online]. Available: <https://github.com/aitorzip/DeepGTAV>
- [44] D. Loiacono, L. Cardamone, and P. L. Lanzi, "Simulated car racing championship: Competition software manual," Apr. 2013, *arXiv: 1304.1672*.
- [45] Mars Auto, "Europilot," 2017. [Online]. Available: <https://github.com/marsauto/europilot>
- [46] A. Amini *et al.*, "Learning robust control policies for end-to-end autonomous driving from data-driven simulation," *IEEE Robot. Autom. Lett.*, vol. 5, no. 2, pp. 1143–1150, 2020.
- [47] M. Bojarski *et al.*, "End to end learning for self-driving cars," *Comput. Res. Repository*, 2016.
- [48] M. Bojarski *et al.*, "Explaining how a deep neural network trained with end-to-end learning steers a car," Apr. 2017, *arXiv:1704.07911*.
- [49] Z. Chen and X. Huang, "End-to-end learning for lane keeping of self-driving cars," in *Proc. IEEE Intell. Veh. Symp.*, 2017, pp. 1856–1860.
- [50] V. Rausch, A. Hansen, E. Solowjow, C. Liu, E. Kreuzer, and J. K. Hedrick, "Learning a deep neural net policy for end-to-end control of autonomous vehicles," in *Proc. IEEE Amer. Control Conf.*, 2017, pp. 4914–4919.
- [51] J. Jung, I. Bae, J. Moon, T. Kim, J. Kim, and S. Kim, "End-to-end steering controller with CNN-based closed-loop feedback for autonomous vehicles," in *Proc. IEEE Intell. Veh. Symp.*, 2019, pp. 617–622.
- [52] L. Caltagirone, M. Bellone, L. Svensson, and M. Wahde, "LiDAR-based driving path generation using fully convolutional neural networks," in *Proc. IEEE Conf. Intell. Transp. Syst.*, 2017, pp. 1–6.
- [53] M. Toromanoff, E. Wirbel, F. Wilhelm, C. Vejarano, X. Perrotton, and F. Moutarde, "End to end vehicle lateral control using a single fisheye camera," in *Proc. IEEE Conf. Intell. Robots Syst.*, 2018, pp. 3613–3619.
- [54] C. Hubschneider, A. Bauer, M. Weber, and J. M. Zöllner, "Adding navigation to the equation: Turning decisions for end-to-end vehicle control," in *Proc. IEEE 20th Int. Conf. Intell. Transp. Syst.*, 2017, pp. 1–8.
- [55] A. I. Maqueda, A. Loquercio, G. Gallego, N. Garcia, and D. Scaramuzza, "Event-based vision meets deep learning on steering prediction for self-driving cars," in *Proc. IEEE Conf. Comput. Vision Pattern Recognit.*, 2018, pp. 5419–5427.

- [56] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vision Pattern Recognit.*, 2016, pp. 770–778.
- [57] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE Conf. Comput. Vision Pattern Recognit.*, 2009, pp. 248–255.
- [58] V. Nair and G. E. Hinton, "Rectified linear units improve restricted boltzmann machines," in *Proc. Int. Conf. Mach. Learn.*, 2010, pp. 807–814.
- [59] H. M. Eraqi, M. N. Moustafa, and J. Honer, "End-to-end deep learning for steering autonomous vehicles considering temporal dependencies," in *Proc. Mach. Learn. Intell. Transp. Syst. Workshop*, 2017.
- [60] X. Zhu, P. Sobhani, and H. Guo, "Long short-term memory over recursive structures," in *Proc. Int. Conf. Mach. Learn.*, 2015, pp. 1604–1612.
- [61] S. Song, X. Hu, J. Yu, L. Bai, and L. Chen, "Learning a deep motion planning model for autonomous driving," in *Proc. IEEE Intell. Veh. Symp.*, 2018, pp. 1137–1142.
- [62] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *Proc. Int. Conf. Learn. Representat.*, 2015.
- [63] F. Yu and V. Koltun, "Multi-scale context aggregation by dilated convolutions," in *Proc. Comput. Res. Repository*, 2015.
- [64] T. Fernando, S. Denman, S. Sridharan, and C. Fookes, "Going deeper: Autonomous steering with neural memory networks," in *Proc. IEEE Int. Conf. Comput. Vision Workshops*, 2017, pp. 214–221.
- [65] T. Fernando, S. Denman, A. McFadyen, S. Sridharan, and C. Fookes, "Tree memory networks for modelling long-term temporal dependencies," *Neurocomputing*, vol. 304, 2018.
- [66] P. Penkov, V. Sriram, and J. Ye, "Applying techniques in supervised deep learning to steering angle prediction in autonomous vehicles," CS229, Stanford University, Tech. Rep., 2016. [Online]. Available: <http://cs229.stanford.edu/proj2016/report/SriramPenkovYe-ApplyingTechniquesInSupervisedDeepLearningToSteeringAnglePredictionInAutonomousVehicles-report.pdf>
- [67] Z. Yang, Y. Zhang, J. Yu, J. Cai, and J. Luo, "End-to-end multi-modal multi-task vehicle control for self-driving cars with visual perceptions," in *Proc. Int. Conf. Pattern Recognit.*, 2018, pp. 2289–2294.
- [68] S. Chowdhuri, T. Pankaj, and K. Zipser, "MultiNet: Multi-modal multi-task learning for autonomous driving," in *Proc. IEEE Winter Conf. Appl. Comput. Vision*, 2019, pp. 1496–1504.
- [69] F. Codevilla, M. Müller, A. López, V. Koltun, and A. Dosovitskiy, "End-to-end driving via conditional imitation learning," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2018, pp. 4693–4700.
- [70] F. Codevilla, E. Santana, A. M. Lopez, and A. Gaidon, "Exploring the limitations of behavior cloning for autonomous driving," in *Proc. IEEE Int. Conf. Comput. Vision*, 2019, pp. 9328–9337.
- [71] A. Amini, W. Schwarting, G. Rosman, B. Araki, S. Karaman, and D. Rus, "Variational autoencoder for end-to-end control of autonomous driving with novelty detection and training de-biasing," in *Proc. Int. Conf. Intell. Robots Syst.*, 2018, pp. 568–575.
- [72] S. Kullback and R. A. Leibler, "On information and sufficiency," *Ann. Math. Statist.*, vol. 22, no. 1, pp. 79–86, 1951.
- [73] P. Wenzel, Q. Khan, D. Cremers, and L. Leal-Taixé, "Modular vehicle control for transferring semantic information between weather conditions using GANs," in *Proc. Conf. Robot Learn.*, 2018, pp. 253–269.
- [74] I. Goodfellow *et al.*, "Generative Adversarial Nets," in *Proc. Neural Inf. Process. Syst.*, 2014, pp. 2672–2680.
- [75] J.-Y. Zhu, T. Park, P. Isola, and A. A. Efros, "Unpaired image-to-image translation using cycle-consistent adversarial networks," in *Proc. IEEE Int. Conf. Comput. Vision*, 2017, pp. 2242–2251.



Abdoulaye O. Ly received the bachelor's degree from Université de Thiès, Thiès, Senegal. He is currently working toward the master's degree in computer science and a mitacs research intern with Université de Moncton, Moncton, NB, Canada. He interned between May and September 2019 as a Software Development Engineer at Mozilla, Toronto, Canada. His research interests include autonomous driving, deep learning, computer vision, and software engineering.



Moulay Akhloufi (Senior Member, IEEE) received the M.Sc. degree in electrical engineering from Ecole Polytechnique de Montreal, Montreal, QC, Canada, the MBA degree from Laval University, Quebec City, QC, Canada, and the Ph.D. degree in electrical engineering from Laval University Quebec City, QC, Canada. He is currently a Professor in Computer Science with the University of Moncton, Moncton, NB, Canada and the Head of the Perception, Robotics, and Intelligent Machines Group. Before joining the University of Moncton in 2016, he worked in the industry and technology transfer in the areas of computer vision and robotics for more than 15 years. He is a member of SPIE.